MATLAB SIMULINK FOR CONTROL

EXPERIMENT NUMBER 10

OBJECTIVES:

- Simulate control systems through MATLAB Simulink.
- Construct second-order system models and observe and analyse the system's response.

APPARATUS:

A PC or Laptop with MATLAB Simulink installed.

THEORY AND DIAGRAMS:

Control System Simulation with Simulink:

Control systems play a pivotal role in regulating and influencing the behaviour of dynamic systems. Simulating these systems provides a valuable means to study and analyse their responses under different conditions.

Control Systems and Simulink:

Control systems are crucial in engineering to manage and manipulate the behaviour of dynamic systems. Simulink, a graphical programming environment in MATLAB, facilitates the modelling, simulation, and analysis of these systems. It offers an intuitive platform for designing and visualising control system models.

Second-Order System Model:

A second-order system is characterised by two poles in its transfer function, representing its dynamic response. In this experiment, we will construct a second-order system model using Simulink. The model's parameters, such as damping ratio and natural frequency, significantly influence the system's behaviour.

Step Input and System Response:

A step input is a fundamental test signal used to analyse the dynamic response of a system. When applied to a control system, it induces a sudden change, allowing us to observe how the system reacts over time. We can visualise and analyse the system's response to a step input through Simulink, gaining insights into its stability and transient characteristics.

Practical Scenario

Solar tracking is a technology designed to enhance the efficiency of solar energy capture by aligning solar panels or mirrors with the sun's position throughout the day. Unlike fixed solar

installations, which remain in a static position, solar tracking systems dynamically adjust the orientation of solar components to maximise exposure to sunlight, as shown in Figure 1.

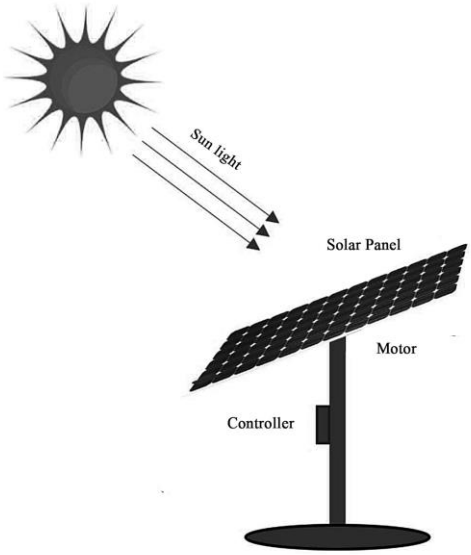


Figure 1

Motion of the Solar Panel:

The rotational motion of the solar panel can be described using the second-order differential equation (1) below.

$$\frac{d^2\theta}{dt^2} = \frac{1}{J}(T - K_d \frac{d\theta}{dt}) \tag{1}$$

Where,

- $T = K_g K_t i$
- $\frac{d^2\theta}{dt^2}$: Second derivative of the tracking angle with respect to time, representing the rotational dynamics of the solar panel.
- J: Moment of inertia of the solar panel. It quantifies the distribution of mass around the axis of rotation.
- K_d: Damping constant. It represents the damping or resistive forces acting against the rotation of the solar panel.
- T: Torque of the motor
- K_t : Torque constant, relating the motor current to the torque produced.

Electrical Dynamics of the Motor:

The electrical dynamics of the motor can be described using the first-order linear differential equation (2) below.

$$\frac{di}{dt} = \frac{1}{L}\ (V - K_g K_f \frac{d\theta}{dt} - Ri) \tag{2}$$

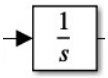
- $\frac{di}{dt}$: Rate of change of current with respect to time, representing the electrical dynamics of the motor.
- L: Inductance of the motor circuit. It measures the ability of the coil to induce a voltage in itself when the current through it changes.
- V: Applied voltage to the motor.
- K_g: Gain constant. It represents the motor gain, which relates the electrical input to the mechanical output.
- K_f : Feedback constant. It accounts for any feedback mechanism influencing the motor dynamics.
- $\frac{d\theta}{dt}$: Rate of change of the tracking angle with respect to time. It's a coupling term indicating that the motor's current is influenced by the rate of change of the tracking angle.
- *R*: Resistance in the motor circuit.

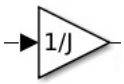
PID Control Systems:

PID (Proportional-Integral-Derivative) control optimises solar trackers by continuously adjusting panel orientation for maximum sunlight exposure. The proportional term (P) responds to current errors, integral (I) eliminates steady-state error over time, and derivative (D) anticipates future errors, preventing overshooting. This dynamic PID algorithm ensures precise tracking as the sun moves across the sky. It enhances tracking accuracy, adapts to environmental changes, and maximises energy harvesting. The result is an efficient and responsive solar tracker, crucial for optimising energy capture in solar power systems and contributing to increased overall energy efficiency.

PROCEDURE:

Part 01

1. Start MATLAB.
2. Start Simulink. In the simulation start window, create a “Blank Module”.
3. Save your Simulink project as “part_01.slx” in a folder created with the name “EC_20XX_XXX”. Also, create a Word or text file and rename it with your student number. Save it in the same folder.
4. Design the Simulink model to control the motion of the solar panel as discussed in Figure 1, referring to equation (1). Follow the instructions below when designing the model.
   a. For the input use Sources → Constant and change the value for 10. This is the torque we are giving as the input to the system for now. We will change this in the next part.
   b. Use the Integrator block  to integrate and get the velocity and position from acceleration.

c. For the Damping constant, use the value Kd in the gain block. 

Define the value for Kd by clicking the three dots and clicking Kd**:** Create. For value, enter 5 and for the location, Base Workspace.

d. Follow the steps 4(b) for J. For the value, enter 8.6 and for the location Base Workspace.

e. To add a text annotation, click the icon from the left side panel and use the following method of writing to display the equation (1).

\frac{d^2\theta}{dt^2}=\frac{1}{J}\left(T-K_d\frac{d\theta}{dt}\right)

f. Attach two scopes to observe the position and velocity. Select the components and right-click → Create a subsystem from the selection to make a Sub System. The final model for the Motion of the Solar Panel will look as shown in Figure 2.

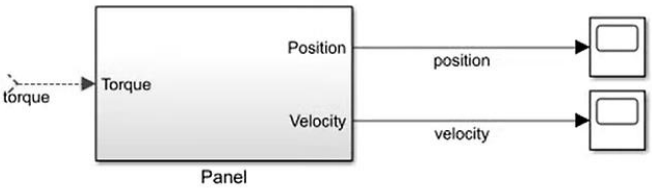


Figure 2

g. Observe the outputs (Use 10 for the simulation stop time) and take screenshots separately. The final scope output of the Model will look like Figure 3. Comment on the behaviour of the solar panel motion on the discussion page.

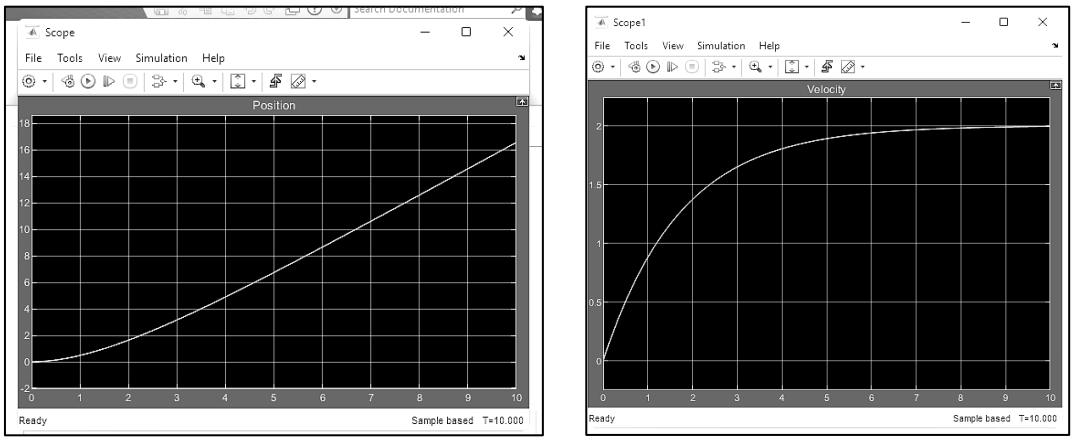


Figure 3

Part 02

1. Design the Simulink model for the Electrical Dynamics of the Motor referring to the equation (2). Follow the instructions mentioned below when designing the model.

a. For the values in gain blocks, use the following. For the location, select Base Workspace.

L: **1.0000e-05**
Kg: **2000**
Kf: **0.0700**
R: **10**

b. Make a subsystem using the knowledge gained from the first part, and your final model will look like Figure 4.

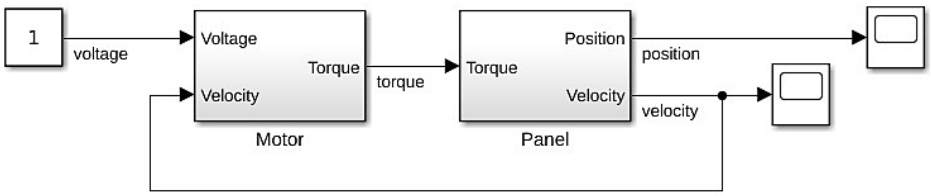


Figure 4

c. Save your model frequently in the folder you created earlier.

2. For the efficiency and accuracy of the solar tracker, it is necessary to install a PID controller in the motor. Since we already have the position of the solar panel, we need to input the position of the sun to find the error. For that, insert a Step Function (Unit step), add a Sum block, and change the sign list to **|+-**. Add a PID controller block from the library browser and install it, as shown in Figure 5.

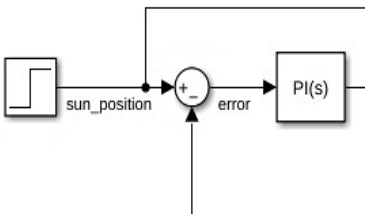


Figure 5

3. Double-click the PID controller and change the values, as shown in Figure 6. Remember to select PI from the controller as we don't need the D term because we don't need a quick response (Usually, the D term refers to rapid changes) as the sun moves across the sky at a considerably low speed. It can be changed as we want to tune up the controller.
4. The final model will look as shown in Figure 7. The position scope will be used to observe the sun and solar panel positions.
5. Run the simulation and use the scopes to observe the model outputs, which should look like Figure 8.

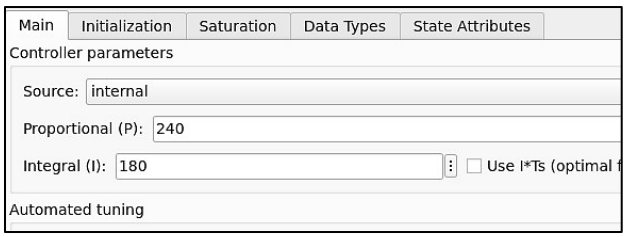


Figure 6

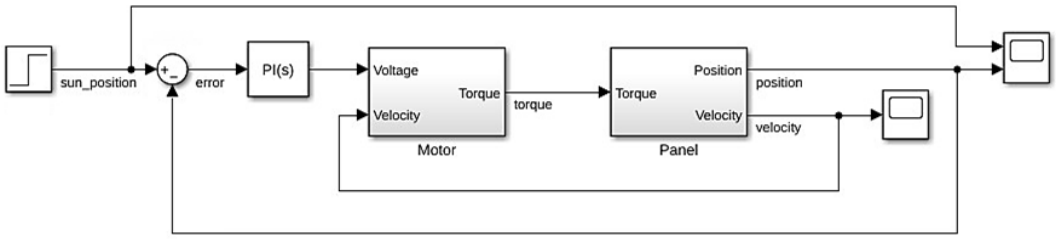


Figure 7

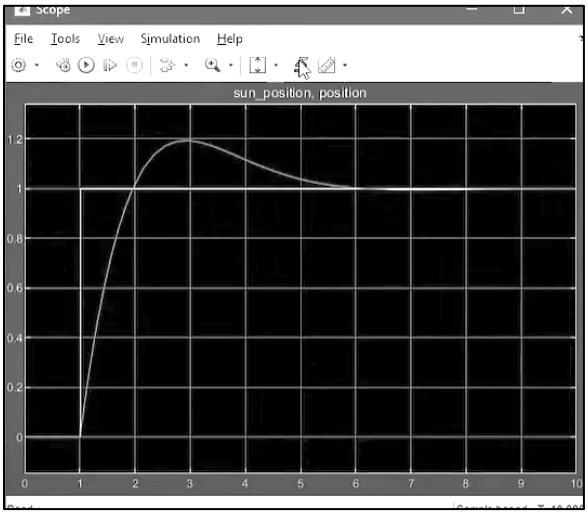


Figure 8

Optional Part

1. In the above part, we have used a step input to simulate the model as the sun data. We can use actual sun data for the sun position to test model behaviour with the input. Go to eKel and download the sunPositionData.mat file.
2. Type the following code in the MATLAB command window. Change 'path of the file' to the actual path of the sunPositionData.mat file.
   ```
   load('path of the file\sunPositionData.mat')
   ```
3. You can plot and observe the graph using the following code.
   ```
   plot(sunTime,rad2deg(sunPosition))
   ```
4. Replace the step function block with an import block.

5. Select the import block → Go to modelling → model settings → Data import/export from the left-hand panel. Tick the Input button and type [sunTime,sunPosition] in the text box as shown in Figure 9.

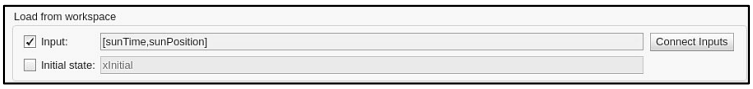


Figure 9

6. In the sunPositionData.mat, we only have 15 hours of data. Therefore, apply the stop time as 15*3600 to observe how the developed model behaves with the provided actual sun data.
7. Run the simulation and observe the plot, as shown in Figure 10.

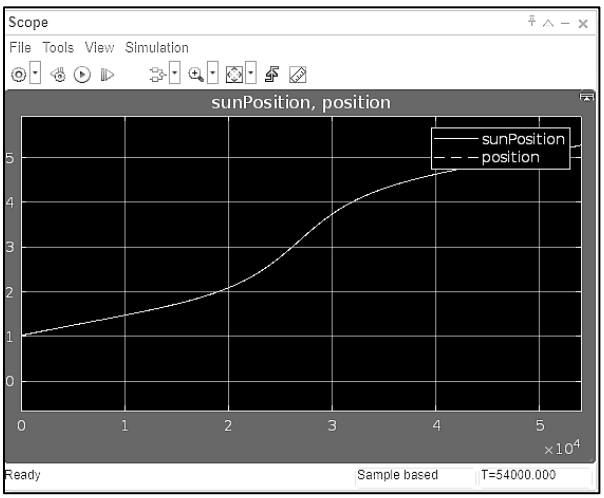


Figure 10

DISCUSSION:

Write your own discussion.